CMSC 441

Paper 2

Due Date: Monday November 2, 11:59PM

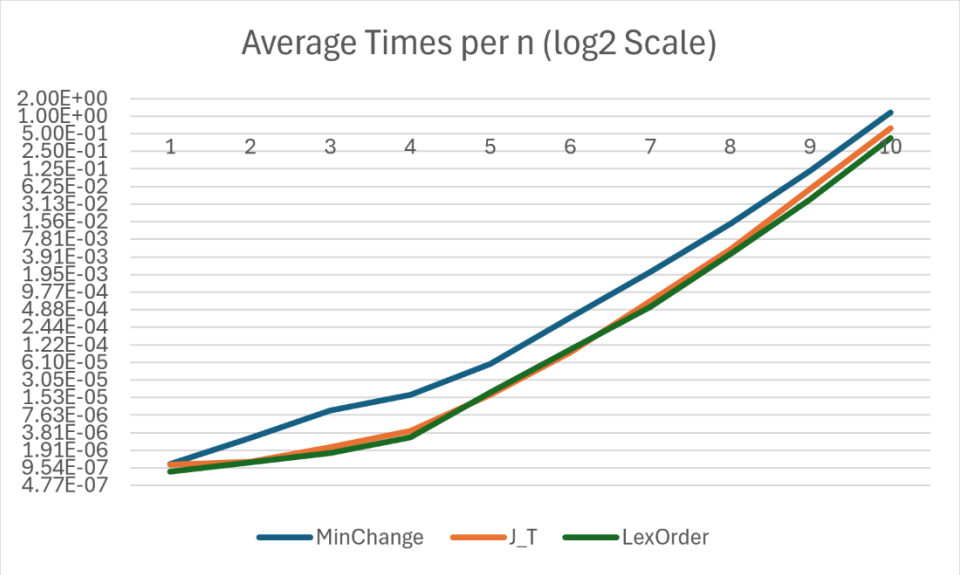
Part A

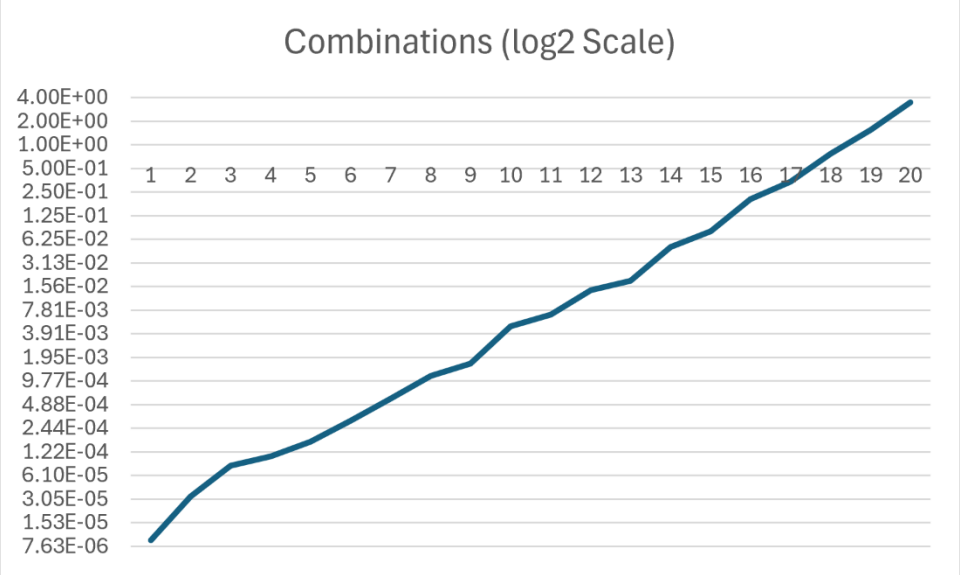
The goal of this exercise is to compare three algorithms for generating permutations of 1, 2, …*n*. Implement the minimal change algorithm we discussed in class (Levitin, page 144), the Johnson and Trotter algorithm (page 145) and the lexicographic order algorithm (Levitin, page 146). (a) Explain, in your own words, how each algorithm works and justify the time efficiency class of each. Include your code. (b) Choose a programming language for these algorithms but note that all the algorithms must be in the same language. Run all three programs and measure the time taken. Increase the value of *n* to as much as you can on your laptop. Time the algorithms and draw a graph showing running time of the algorithms on the same graph. (c) Which algorithm is more efficient? Write a note about the pros and cons of each algorithm. Justify your choice of the programming language. What features of the language were useful for these algorithms?

(a)

a. Minimal Change: Elements will be inserted from right to left, being swapped in the same direction until reaching the end. If the element needs to be inserted into multiple different permutations, alternate inserting right to left and left to right. The time is n! as there are n! permutations, with each permutation needing either an insertion or a swap, both being constant time.
b. Johnson-Trotter: This algorithm works by putting a flag on each element pointing left initially, then moving the largest element pointing to a smaller element until it cannot be moved any more. Then reverse the direction of all elements that are larger than k. Keep doing this until there are no elements that point to smaller elements. The time efficiency of this algorithm is n! as there are n! permutations, and for most permutations, there is a required reversal of some amount of the list, which would result in n!*n, which is n!.
c. Lexicographic Order: This algorithm gives all permutations in order of smallest to largest. It does this by swapping the first element from the right that is smaller than the element to its right with the first element larger than it from the right side. Each time this swap occurs, all elements to the right of the left side of the swap are

reversed. The time efficiency of this is n! as there are n! permutations; however, this algorithm requires reversals, which results in the time being about n!*n.





(b) Python

(c)

a. Minimal Change
    i. Pros: N/A
    ii. Cons: Very large algorithm to write, requires multiple arrays
b. J-T
    i. Pros: Almost as efficient as Lexicographic order
    ii. Cons: Decent size/complexity of the algorithm, requires an extra array
c. Lex Order
    i. Pros: Most efficient algorithm, only requires one array

ii. Cons: N/A

d. Why Python?

i. I chose python because I am familiar with python, and the timeit class is very useful for running a function multiple times to get a good average.

Part B

We discussed in class how to generate the set of all subsets and the set of *all* permutations. But we did not discuss how to generate combinations. (a) For this exercise, write code (in a language of your choice) to generate all *i* combinations of *n* items, for a given value of *n* and *i*. (b) How many are there? (Essentially what is the value of *n* choose *i*.) Justify your answer. (c) Explain how the algorithm works. Assume you are given the first *n* numbers as input and you are given *i* as input. (You can fix *n* to something like 10 to begin with and give *i* as *n*/2.) (d) Run your code using larger and larger values of *n*. Show a graph of how the time taken by the algorithm increases as a function of *n*. (for the graph itself, you can fix *i* to be *n*/2). What should be the theoretical time efficiency of the algorithm? What is your finding about the actual running time?

In practice, this algorithm has many applications. We will see some of them later.

(a) A

(b) All combinations I of n is equal to n!/i!*(n-i)!. There are a total of n! permutations, dividing by i! removes all repeats, and dividing by (n-i)! removes all permutations that aren't of size i.

(c) This algorithm works by taking a number n and turning it into an array from 0-9. Then, this algorithm chooses either to take the first element of the array, or to not take the first element of the array. Either way, it skips to the next element in the array. It does this until the new array is the size of i, or the original array is empty.

(d) The time efficiency should be n!/i!*(n-i)! + n!/(i-1)!*(n-i+1)!
The actual time efficiency is about n!

(e) Github link: https://github.com/roycenogales/Permutations-Analysis